

Loan Prediction - ML Report

Decision Tree & Random Forest | Manual Metrics Implementation

1. Project Overview

This project predicts loan approval (Y/N) using Decision Tree and Random Forest. All metrics (accuracy, precision, recall) and prediction logic are implemented manually without using sklearn's built-in metric or predict functions.

2. Data Preprocessing — preprocess(df)

Cleans both train and test data before modelling.

Step	What it does
Fill missing — categorical	Uses mode (most common value) to fill blanks.
Fill missing — numeric	Uses mean to fill blanks.
Encode categories	Converts text columns (Gender, Education etc.) to numbers using category codes.

3. Manual Functions (No sklearn metrics)

These replace all sklearn built-in metric and predict functions.

3.1 manual_predict(model, X_test)

Gets probabilities from predict_proba(), then manually loops to find the class with highest probability.

```
proba = model.predict_proba(X_test) # e.g. [[0.3, 0.7], [0.6, 0.4]] classes =  
model.classes_  
for row in proba: max_index = 0  
    for i in range(len(row)): if row[i] > row[max_index]: max_index = i  
predictions.append(classes[max_index])
```

3.2 manual_accuracy(y_true, y_pred)

Counts how many predictions match actual values, divides by total.

```
correct = 0  
for actual, predicted in zip(y_true, y_pred): if actual == predicted: correct += 1  
return correct / len(y_true) # Formula: correct / total
```

3.3 manual_precision(y_true, y_pred)

Precision = $TP / (TP + FP)$. Counts true positives and false positives by looping.

```
tp = 0 fp = 0
for actual, predicted in zip(y_true, y_pred):
    if predicted == 1: # model said YES
        if actual == 1: tp += 1 # correctly said YES
        else: fp += 1 # wrongly said YES
return tp / (tp + fp)
```

3.4 manual_recall(y_true, y_pred)

Recall = $TP / (TP + FN)$. Counts true positives and false negatives by looping.

```
tp = 0 fn = 0
for actual, predicted in zip(y_true, y_pred):
    if actual == 1: # actually YES
        if predicted == 1: tp += 1 # correctly found
        else: fn += 1 # missed it
return tp / (tp + fn)
```

Metric	Formula
Accuracy	Correct predictions / Total predictions
Precision	$TP / (TP + FP)$
Recall	$TP / (TP + FN)$

4. Decision Tree Model — Task 01

A single tree that splits data on feature thresholds. `max_depth` limits how deep it grows.

4.1 Class Methods

Method	What it does
<code>__init__(max_depth)</code>	Creates the sklearn tree internally. <code>max_depth</code> controls complexity.
<code>train(X_train, y_train)</code>	Calls <code>model.fit()</code> — tree learns split rules from training data.
<code>predict(X_test)</code>	Calls <code>manual_predict()</code> — uses probabilities, NOT <code>model.predict()</code> .
<code>evaluate(y_test, y_pred)</code>	Calls <code>manual_accuracy</code> , <code>manual_precision</code> , <code>manual_recall</code> .

4.2 Experiment Results

Depth	Accuracy	Precision	Recall
2	0.7886	0.7596	0.9875
5	0.7398	0.7449	0.9125
None	0.6829	0.7531	0.7625

Depth=2 gives best accuracy. Depth=None overfits — memorises training data, fails on new data.

5. Random Forest Model — Task 02

Builds many trees, each on a random data subset. Final prediction = majority vote across all trees. More stable than a single Decision Tree.

5.1 Class Methods

Method	What it does
<code>__init__(n_estimators)</code>	Sets number of trees. More trees = more stable result.
<code>train(X_train, y_train)</code>	Fits all trees. Each tree sees a random sample of the data.
<code>predict(X_test)</code>	Calls <code>manual_predict()</code> — aggregates probabilities, picks top class manually.
<code>evaluate(y_test, y_pred)</code>	Same manual metrics: accuracy, precision, recall.

5.2 Experiment Results

n_estimators	Accuracy	Precision	Recall
10	0.7154	0.7368	0.8750
50	0.7561	0.7500	0.9375
100	0.7642	0.7525	0.9500

100 estimators gives best accuracy. Adding more trees beyond 100 gives diminishing returns.

6. Submission

Best model (Random Forest, n=100) predicted loan status for all 367 test applicants. Saved to `submission.csv` with columns: `Loan_ID`, `Loan_Status` (Y/N).